

Programação Orientada a Aspectos com Spring

Disciplina: Arquitetura e Desenvolvimento Web

Tema: AOP com Spring no projeto produtos-api

Objetivos da aula

- Entender o problema que a AOP resolve
- Dominar os conceitos de Aspect, Join Point, Pointcut e Advice
- Compreender como o Spring aplica aspectos via proxy
- Ver um exemplo prático no backend da turma

O problema sem AOP

Interesses transversais aparecem em vários pontos:

- Logging
- Auditoria
- Monitoramento
- Segurança
- Tratamento de exceções

Resultado comum:

- Scattering (código espalhado)
- Tangling (responsabilidades misturadas)

Ideia central da AOP

Separar preocupações transversais da regra de negócio.

Sem AOP:

- ProdutoService = negócio + log + monitoramento

Com AOP:

- ProdutoService = negócio
- ObservabilidadeAspect = log + tempo + status

Conceitos fundamentais

- Aspect: classe com comportamento transversal
- Join Point: ponto de execução (método)
- Pointcut: filtro dos pontos a interceptar
- Advice: código executado no ponto interceptado

Tipos de advice mais usados:

- Before, After, AfterReturning, AfterThrowing, Around

Por que Around é muito usado?

Com Around você consegue:

- Medir tempo antes e depois do método
- Registrar sucesso e falha
- Repassar exceções sem alterar regra de negócio

Padrão comum de observabilidade:

- INICIO -> executa metodo -> FIM com tempo/status

Spring AOP na pratica

No Spring, aspectos são aplicados em beans gerenciados:

- Service
- Component
- Repository
- RestController

Dependencia usada neste projeto (Spring Boot 4):

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-aspectj</artifactId>  
</dependency>
```

Limitações importantes

Spring AOP (proxy-based):

- Intercepta métodos de beans Spring
- Não intercepta objetos criados com new
- Não intercepta construtores, campos e métodos private

Armadilha comum:

- Self-invocation pode não passar pelo proxy

Exemplo no backend da turma

Arquivo criado:

- backend/produtos-api/src/main/java/br/unifor/produtosapi/aspect/ObservabilidadeAspect.java

Pointcut adotado:

- controller e service

Log gerado:

- INICIO metodo=... args=...
- FIM metodo=... tempoMs=... sucesso=true/false

Trecho do aspecto

```
@Pointcut("within(br.unifor.produtosapi.controller..*) || within(br.unifor.produtosapi.service..*)")
public void camadasApi() {}

@Around("camadasApi()")
public Object registrarTempoEStatus(ProceedingJoinPoint joinPoint) throws Throwable {
    // início
    // proceed
    // fim com tempo e status
}
```

Demo em sala (5 minutos)

1. Subir backend

```
cd backend/produtos-api  
./mvnw spring-boot:run
```

2. Chamar endpoint

```
curl http://localhost:8080/tipos
```

3. Mostrar no console o log do aspecto

Quando usar e quando evitar

Use AOP para:

- Logging técnico
- Auditoria
- Métricas e monitoramento

Evite AOP para:

- Regra de negócio principal
- Cálculo de domínio
- Decisões centrais de caso de uso

Perguntas de fixação

1. O que é interesse transversal?
2. Qual diferença entre scattering e tangling?
3. O que diferencia pointcut de advice?
4. Por que Around é útil para observabilidade?
5. Quando AOP não intercepta?

Fechamento

AOP no Spring melhora organização arquitetural quando aplicada em preocupações transversais.

No projeto da turma, ela reduz código repetido e melhora rastreabilidade sem poluir services/controllers.